

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Отчет по дисциплине: «Вычислительная математика»

«Лабораторная работа №1».

Студент,
группы 5130201/40003

_____ Адиатуллин Т. Р.

Руководитель,
Преподаватель

_____ Пак В. Г.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1	Постановка задач	3
2	Задание 1	4
2.1	Нахождение значения функции с помощью интерполяционного многочлена Лагранжа	4
2.2	Нахождение значения функции с помощью схемы Эйткена	5
3	Задание 2	7
3.1	Вычисление значения функции с помощью второго многочлена Ньютона	7
3.2	Вычисление значения функции с помощью первого многочлена Ньютона	8
4	Задание 3	10
4.1	Вычисление значения функции с помощью первого многочлена Ньютона	10
4.2	Вычисление значения функции с помощью второго многочлена Ньютона	11
4.3	Вычисление значения функции с помощью многочлена Бесселя	12

1 Постановка задач

Для данной табличной функции вычислить приближённое значение в точке интерполяции с помощью многочлена Лагранжа и схемы Эйткена.

Для табличной функции из задания 1 вычислить приближённые значения в точках интерполяции с помощью подходящего многочлена Ньютона с разделёнными разностями.

Для данной табличной функции вычислить приближённые значения в точках интерполяции с помощью подходящего многочлена Ньютона с конечными разностями (точки $x^{(1)}$, $x^{(2)}$) и наиболее подходящего из многочленов Гаусса, Стирлинга или Бесселя (точка x).

2 Задание 1

Даны несколько точек x некоторой функции и соответствующие им значения y . Можно увидеть эти значения в приведённой ниже таблице 1.

i	x	y
0	0,32	1,4126
1	0,37	2,5210
2	0,41	4,1020
3	0,45	3,1100
4	0,61	3,0980
5	0,65	2,7110

Таблица 1: Таблица значений функции в различных точках.

2.1 Нахождение значения функции с помощью интерполяционного многочлена Лагранжа

В первой части задания требуется найти приближённое значение функции в заданной точке интерполяции с использованием многочлена Лагранжа. Общий вид многочлена Лагранжа задаётся формулой

$$L_n(x) = \sum_{i=0}^n y_i \cdot l_{n,i}(x), \quad l_{n,i}(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

Используя значения из таблицы, вычислим приближённое значение функции в точке $x = 0,39$. Подставим значения узлов интерполяции:

$$L_5(0,39) = 1,4126 \cdot \frac{(0,39 - 0,37)(0,39 - 0,41)(0,39 - 0,45)(0,39 - 0,61)(0,39 - 0,65)}{(0,32 - 0,37)(0,32 - 0,41)(0,32 - 0,45)(0,32 - 0,61)(0,32 - 0,65)} + \\ + 2,5210 \cdot \frac{(0,39 - 0,32)(0,39 - 0,41)(0,39 - 0,45)(0,39 - 0,61)(0,39 - 0,65)}{(0,37 - 0,32)(0,37 - 0,41)(0,37 - 0,45)(0,37 - 0,61)(0,37 - 0,65)} + \dots \approx 3,579.$$

В листинге 1 приведён программный код, реализующий вычисление.

```
1 import numpy as np
2
3 # Данные
4 x_vals = np.array([0.32, 0.37, 0.41, 0.45, 0.61, 0.65])
5 y_vals = np.array([1.4126, 2.521, 4.102, 3.110, 3.098, 2.711])
6
7 # Функция для вычисления интерполяционного многочлена Лагранжа
8 def lagrange_interpolation(x, x_vals, y_vals):
9     n = len(x_vals)
```

```

10     result = 0
11     for i in range(n):
12         term = y_vals[i]
13         for j in range(n):
14             if j != i:
15                 term *= (x - x_vals[j]) / (x_vals[i] - x_vals[j])
16         result += term
17     return result
18
19 x_to_interpolate = 0.39
20 y_interpolated = lagrange_interpolation(x_to_interpolate, x_vals,
    y_vals)
21 print(f"Значение функции в точке x = {x_to_interpolate} через
    многочлен Лагранжа: {y_interpolated}")

```

Листинг 1: Вычисление значения функции с помощью многочлена Лагранжа

В итоге получаем значение $y(0,39) = 3,579$.

2.2 Нахождение значения функции с помощью схемы Эйткена

Вычислительная схема Эйткена используется для последовательного рекуррентного вычисления значения интерполяционного многочлена Лагранжа L_n в заданной точке x . Для нахождения значения функции по методу Эйткена применяется соответствующая формула:

$$P_{i,i+1}(x) = \frac{(x - x_i) \cdot P_{i+1}(x) - (x - x_{i+1}) \cdot P_i(x)}{x_{i+1} - x_i}.$$

Для первых двух точек расчёт значения в точке $x = 0,39$ имеет вид:

$$P_{0,1}(0,39) = \frac{(0,39 - 0,32) \cdot 2,5210 - (0,39 - 0,37) \cdot 1,4126}{0,37 - 0,32} = 2,96396.$$

Далее значения пересчитываются для пар более высоких порядков, пока не будет получен $P_{0,1,2,3,4,5}(0,39)$.

Значение функции в точке $x = 0,39$ было получено с помощью программной реализации, представленной на листинге 2.

```

1 def aitken_interpolation(x, x_vals, y_vals):
2     n = len(x_vals)
3     A = np.zeros((n, n))
4     for i in range(n):
5         A[i][0] = y_vals[i]
6     for j in range(1, n):
7         for i in range(n - j):
8             A[i][j] = ((x - x_vals[i]) * A[i+1][j-1] - (x - x_vals
    [i+j]) * A[i][j-1]) / (x_vals[i+j] - x_vals[i])
9     return A[0][n-1]

```

```
10 |
11 | y_aitken = aitken_interpolation(x_to_interpolate, x_vals, y_vals)
12 | print(f"Значение функции в точке x = {x_to_interpolate} через
    | схему Эйткена: {y_aitken}")
```

Листинг 2: Вычисление значения функции с помощью схемы Эйткена

В результате вычислений было получено то же значение, что и при использовании интерполяционного многочлена Лагранжа: $y(0,39) = 3,579$.

3 Задание 2

В этом задании нужно по таблице 1 через интерполяционные многочлены Ньютона с разделёнными разностями вычислить значения в точках $x^{(1)} = 0,66$ и $x^{(2)} = 0,30$.

3.1 Вычисление значения функции с помощью второго многочлена Ньютона

Для вычисления значения функции в точке $x^{(1)} = 0,66$ используется интерполяционный многочлен Ньютона с разделёнными разностями второго вида (интерполирование назад). Его общий вид задаётся формулой:

$$P_n(x) = f(x_n) + f(x_{n-1}, x_n)(x - x_n) + f(x_{n-2}, x_{n-1}, x_n)(x - x_n)(x - x_{n-1}) + \dots$$

В листинге 3 представлен программный код для вычисления.

```
1 import numpy as np
2
3 x_values = np.array([0.32, 0.37, 0.41, 0.45, 0.61, 0.65])
4 y_values = np.array([1.4126, 2.521, 4.102, 3.110, 3.098, 2.711])
5
6 def divided_difference(x, y):
7     n = len(x)
8     diff_table = np.zeros((n, n))
9     diff_table[:, 0] = y
10    for j in range(1, n):
11        for i in range(n - j):
12            diff_table[i, j] = (diff_table[i + 1, j - 1] -
13                               diff_table[i, j - 1]) / (x[i + j] - x[i])
14    return diff_table
15
16 def newton_backward(x_values, y_values, x):
17     diff_table = divided_difference(x_values, y_values)
18     n = len(x_values)
19     result = y_values[-1]
20     term = 1
21     for i in range(1, n):
22         term *= (x - x_values[n - i])
23         result += term * diff_table[n - 1 - i, i]
24     return result
25
26 x_point = 0.66
27 result = newton_backward(x_values, y_values, x_point)
28 print(f"Значение функции в точке x = {x_point}: {result}")
```

Листинг 3: Вычисление значения функции с помощью второго многочлена Ньютона

x_i	$f(x_i)$	$f(x_i, x_{i+1})$	$f(x_i, \dots)$	$f(x_i, \dots)$	$f(x_i, \dots)$	$f(x_i, \dots)$
0.32	1.4126	22.168				
0.37	2.5210	39.525	192.855			
0.41	4.1020	-24.800	-804.062	-7668.59		
0.45	3.1100	-0.075	103.020	3779.50	39476.17	
0.61	3.0980	-9.675	-40.000	-595.91	-21876.55	-185917.33
0.65	2.7110					

Таблица 2: Таблица разделённых разностей.

По результатам работы функции получена таблица разделённых разностей:

Подставим значения нижнего края таблицы (диагонали снизу вверх) в расчётную формулу для точки 0,66:

$$P_5(0,66) = 2,7110 + (-9,675)(0,66 - 0,65) + (-40,000)(0,66 - 0,65)(0,66 - 0,61) + (-595,91)(0,66 - 0,65)(0,66 - 0,61)(0,66 - 0,45) + \dots \approx 0,791.$$

В результате вычислений было получено значение $y(0,66) = 0,791$.

3.2 Вычисление значения функции с помощью первого многочлена Ньютона

Для вычисления значения функции в точке $x^{(2)} = 0,30$ используется интерполяционный многочлен Ньютона с разделёнными разностями первого вида (интерполирование вперёд). Его общий вид задаётся формулой:

$$P_1(x) = f(x_0) + f(x_0, x_1)(x - x_0) + f(x_0, x_1, x_2)(x - x_0)(x - x_1) + \dots$$

В листинге 4 приведён программный код.

```

1 def newton_forward(x_values, y_values, x):
2     diff_table = divided_difference(x_values, y_values)
3     result = y_values[0]
4     term = 1
5     for i in range(1, len(x_values)):
6         term *= (x - x_values[i - 1])
7         result += term * diff_table[0, i]
8     return result
9
10 x_point = 0.30
11 result = newton_forward(x_values, y_values, x_point)
12 print(f"Значение функции в точке x = {x_point}: {result}")

```

Листинг 4: Вычисление значения функции с помощью первого многочлена Ньютона

Используется та же таблица разделённых разностей (Таблица 2). Подставим значения верхней диагонали в расчётную формулу для точки 0,30:

$$P_5(0,30) = 1,4126 + 22,168(0,30 - 0,32) + 192,855(0,30 - 0,32)(0,30 - 0,37) + \\ + (-7668,59)(0,30 - 0,32)(0,30 - 0,37)(0,30 - 0,41) + \dots \approx 4,557.$$

В результате вычислений было получено значение $y(0,30) = 4,557$.

4 Задание 3

В этом задании необходимо, используя функцию $y = \sqrt[3]{x}$, заданную на отрезке от 0,0 до 10,0 с шагом $h = 2,0$, вычислить значения в точках $x^{(1)} = -0,02$ и $x^{(2)} = 10,01$ с помощью интерполяционных многочленов Ньютона с конечными разностями. Кроме того, требуется определить значение функции в точке $x = 5,0$ с использованием одного из многочленов Гаусса, Стирлинга или Бесселя.

i	x	y
0	0,0	0,0000
1	2,0	1,2599
2	4,0	1,5874
3	6,0	1,8171
4	8,0	2,0000
5	10,0	2,1544

Таблица 3: Таблица значений функции $y = \sqrt[3]{x}$ в различных точках.

4.1 Вычисление значения функции с помощью первого многочлена Ньютона

Найдём значения в точке $x^{(1)} = -0,02$ с помощью первого многочлена Ньютона с конечными разностями. Формула имеет вид:

$$P_n(x) = y_0 + q\Delta y_0 + \frac{q(q-1)}{2!}\Delta^2 y_0 + \dots, \quad q = \frac{x - x_0}{h}.$$

```
1 import numpy as np
2
3 x_values = np.array([0.0, 2.0, 4.0, 6.0, 8.0, 10.0])
4 y_values = np.array([0.0000, 1.2599, 1.5874, 1.8171, 2.0000,
5                       2.1544])
6
7 def finite_differences(y):
8     n = len(y)
9     diff_table = np.zeros((n, n))
10    diff_table[:, 0] = y
11    for j in range(1, n):
12        for i in range(n - j):
13            diff_table[i, j] = diff_table[i + 1, j - 1] -
14            diff_table[i, j - 1]
15    return diff_table
16
17 def newton_forward_finite(x_values, y_values, x):
18     h = x_values[1] - x_values[0]
```

```

17 q = (x - x_values[0]) / h
18 diff_table = finite_differences(y_values)
19 result = y_values[0]
20 q_prod = 1
21 for i in range(1, len(x_values)):
22     q_prod *= (q - (i - 1)) / i
23     result += q_prod * diff_table[0, i]
24 return result
25
26 x_point = -0.02
27 result = newton_forward_finite(x_values, y_values, x_point)
28 print(f"Значение функции в точке x = {x_point}: {result:.4f}")

```

Листинг 5: Вычисление значения функции с помощью первого многочлена Ньютона с конечными разностями

По результатам работы функции построена таблица конечных разностей:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$
0.0	0.0000	1.2599				
2.0	1.2599	0.3275	-0.9324			
4.0	1.5874	0.2297	-0.0978	0.8346		
6.0	1.8171	0.1829	-0.0468	0.0510	-0.7836	
8.0	2.0000	0.1544	-0.0285	0.0183	-0.0327	0.7509
10.0	2.1544					

Таблица 4: Таблица конечных разностей.

Для $x = -0,02$, $h = 2,0$, параметр $q = \frac{-0,02-0,0}{2} = -0,01$. Подставляя верхнюю строку разностей:

$$P_5(-0,02) = 0,0000 + (-0,01)(1,2599) + \frac{(-0,01)(-0,01-1)}{2}(-0,9324) + \dots \approx -0,0237.$$

4.2 Вычисление значения функции с помощью второго многочлена Ньютона

Найдём значения в точке $x^{(2)} = 10,01$ с помощью второго многочлена Ньютона с конечными разностями. Формула имеет вид:

$$P_n(x) = y_n + q\Delta y_{n-1} + \frac{q(q+1)}{2!}\Delta^2 y_{n-2} + \dots, \quad q = \frac{x - x_n}{h}.$$

```

1 def newton_backward_finite(x_values, y_values, x):
2     h = x_values[1] - x_values[0]
3     n = len(x_values)
4     q = (x - x_values[-1]) / h
5     diff_table = finite_differences(y_values)
6     result = y_values[-1]

```

```

7     q_prod = 1
8     for i in range(1, n):
9         q_prod *= (q + (i - 1)) / i
10        result += q_prod * diff_table[n - 1 - i, i]
11    return result
12
13 x_point = 10.01
14 result = newton_backward_finite(x_values, y_values, x_point)
15 print(f"Значение функции в точке x = {x_point}: {result:.4f}")

```

Листинг 6: Вычисление значения функции с помощью второго многочлена Ньютона с конечными разностями

Для $x = 10,01$, параметр $q = \frac{10,01-10,0}{2} = 0,005$. Подставляя значения нижней диагонали таблицы разностей:

$$P_5(10,01) = 2,1544 + (0,005)(0,1544) + \frac{0,005(0,005 + 1)}{2}(-0,0285) + \dots \approx 2,1558.$$

4.3 Вычисление значения функции с помощью многочлена Бесселя

В данном случае необходимо найти значение функции в точке $x = 5,0$. Многочлен Бесселя целесообразно использовать по двум причинам. Во-первых, таблица содержит чётное число узлов (6 узлов). Во-вторых, интерполируемая точка $x = 5,0$ находится ровно посередине между центральными узлами интерполяции $x_2 = 4,0$ и $x_3 = 6,0$. Для центра интервала параметр $q = \frac{x-x_0}{h} = \frac{5,0-4,0}{2} = 0,5$. В формуле Бесселя при $q = 0,5$ все нечётные разности умножаются на множитель $(q - 0,5)$, который обращается в нуль, что значительно упрощает вычисления и повышает точность.

Формула имеет вид:

$$\begin{aligned}
 P_n = & \frac{y_0 + y_1}{2} + \left(q - \frac{1}{2}\right) \Delta y_0 + \frac{q(q-1)}{2!} \cdot \frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} + \\
 & + \frac{q(q-1)(q-\frac{1}{2})}{3!} \Delta^3 y_{-1} + \frac{q(q-1)(q+1)(q-2)}{4!} \cdot \frac{\Delta^4 y_{-2} + \Delta^4 y_{-1}}{2}.
 \end{aligned}$$

Примем за $x_0 = 4,0$ (тогда $y_0 = 1,5874$, $y_1 = 1,8171$). Подставим разности из таблицы:

$$\begin{aligned}
 P(5,0) = & \frac{1,5874 + 1,8171}{2} + 0 + \frac{0,5(0,5-1)}{2} \cdot \frac{(-0,0978) + (-0,0468)}{2} + 0 + \dots \\
 = & 1,70225 + (-0,125) \cdot (-0,0723) \approx 1,711.
 \end{aligned}$$

Истинное значение $\sqrt[3]{5} \approx 1,70997$.

В листинге 7 представлен исправленный программный код (с корректными значениями y) для вычисления значения функции в точке $x = 5,0$.

```

1 import math
2
3 # Корректные данные для  $y = \sqrt[3]{x}$ 
4 y_values = np.array([0.0000, 1.2599, 1.5874, 1.8171, 2.0000,
5                       2.1544])
6
7 def compute_deltas(y):
8     n = len(y)
9     deltas = []
10    for i in range(1, n):
11        delta = [y[j + 1] - y[j] for j in range(n - i)]
12        deltas.append(delta)
13    y = delta
14    return deltas
15
16 deltas = compute_deltas(y_values)
17 q = 0.5 # Для  $x = 5.0$ ,  $x_0 = 4.0$ ,  $h = 2.0$ 
18
19 def bessel_polynomial(y_vals, deltas, q, idx_x0):
20     # Упрощенная реализация Бесселя для центральных узлов
21     P = (y_vals[idx_x0] + y_vals[idx_x0+1]) / 2.0
22     P += (q - 0.5) * deltas[0][idx_x0]
23     P += (q * (q - 1) / 2.0) * ((deltas[1][idx_x0-1] + deltas[1][
24     idx_x0]) / 2.0)
25     P += (q * (q - 1) * (q - 0.5) / 6.0) * deltas[2][idx_x0-1]
26     P += (q * (q - 1) * (q + 1) * (q - 2) / 24.0) * ((deltas[3][
27     idx_x0-2] + deltas[3][idx_x0-1]) / 2.0)
28     return P
29
30 result = bessel_polynomial(y_values, deltas, q, 2)
31 print(f"Значение по Бесселю: {result:.4f}")

```

Листинг 7: Вычисление значения функции с помощью многочлена Бесселя

В результате правильного вычисления по многочлену Бесселя для функции $\sqrt[3]{x}$ в точке $x = 5,0$ получаем значение $y(5,0) \approx 1,7110$, что весьма близко к точному значению 1,70997.